



Heaven's Light is Our Guide

Department of Computer Science & Engineering
Rajshahi University of Engineering & Technology, Bangladesh

Course Code: CSE 2103
Course Title: Digital Logic Design

Presented by,
Prof. Dr. Boshir Ahmed
Md. Zahirul Islam

Binary Arithmetic

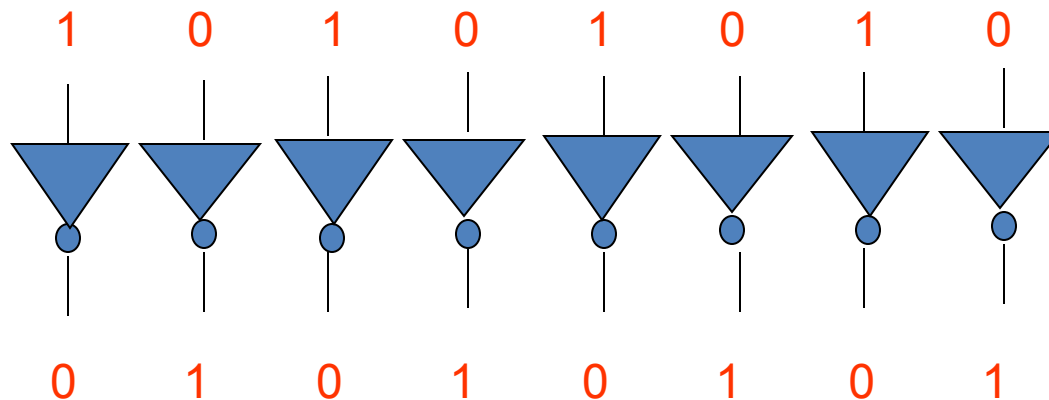
- Binary addition
- Binary subtraction
- Binary multiplication
- Binary division

Complements of Binary Numbers

- 1's complements
- 2's complements

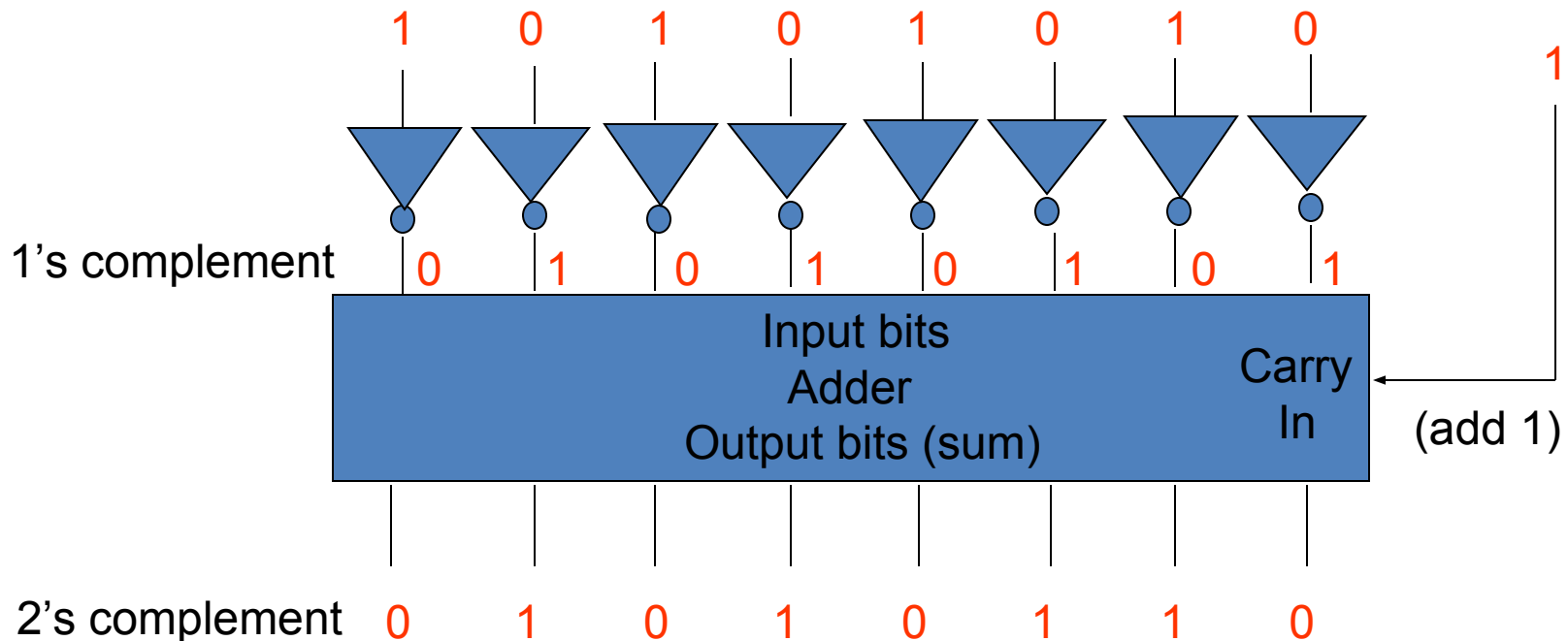
Complements of Binary Numbers

- 1's complement
- Change all 1s to 0s and all 0s to 1s



Complements of Binary Numbers

- 2's complement
- Find 1's complement and then add 1



Signed Numbers

Topics for Signed Numbers

- Signed-magnitude form
- 1's and 2's complement form
- Decimal value of signed numbers
(How to convert)
- Range of values (max and min)
- Floating-point numbers

Signed Numbers

- Signed-magnitude form
 - The sign bit is the left-most bit in a signed binary number
 - A 0 sign bit indicates a positive magnitude
 - A 1 sign bit indicates a negative magnitude

Signed Numbers

- 1's complement form
 - A negative value is the 1's complement of the corresponding positive value
- 2's complement form
 - A negative value is the 2's complement of the corresponding positive value

Signed Numbers

- Decimal value of signed numbers
 - Sign-magnitude
 - 1's complement
 - 2's complement

Signed Numbers

- Range of Values

Total combinations = 2^n

2's complement form:

$$-(2^{n-1}) \text{ to } +(2^{n-1} - 1)$$

Range for 8 bit number:

$$n = 8$$

$$-(2^{8-1}) = -2^7 = -128 \quad \text{minimum}$$

$$+(2^{8-1}) - 1 = +2^7 - 1 = +127 \quad \text{maximum}$$

Total combination of numbers is $2^8 = 256$.

Signed Numbers

Range for 16 bit number:

$$n = 16$$

$$-(2^{16-1}) = -2^{15} = -32768 \quad \text{minimum}$$

$$+(2^{16-1}) - 1 = +2^{15} = +32767 \quad \text{maximum}$$

Total combinations is $2^{16} = 65536$ (64K)

8 bit examples:

$$10000000 = -128$$

$$11111111 = -1$$

$$10000001 = -127$$

$$01111111 = +127$$

Arithmetic Operations with Signed Numbers

- Addition
- Subtraction
- Multiplication
- Division

Arithmetic Operations with Signed Numbers

Addition of Signed Numbers

- The parts of an addition function are:
 - Augend - The first number
 - Addend - The second number
 - Sum - The result

Numbers are always added two at a time.

Arithmetic Operations with Signed Numbers

Four conditions for adding numbers:

1. Both numbers are positive.
2. A positive number that is larger than a negative number.
3. A negative number that is larger than a positive number.
4. Both numbers are negative.

Arithmetic Operations with Signed Numbers

Signs for Addition

- When both numbers are positive, the sum is positive.
- When the larger number is positive and the smaller is negative, the sum is positive. The carry is discarded.

Arithmetic Operations with Signed Numbers

Signs for Addition

- When the larger number is negative and the smaller is positive, the sum is negative (2's complement form).
- When both numbers are negative, the sum is negative (2's complement form). The carry bit is discarded.

Examples (8 bit numbers)

- Add 7 and 4 (both positive)

$$\begin{array}{r}
 00000111 \quad 7 \\
 +00000100 \quad +4 \\
 \hline
 00001011 \quad 11
 \end{array}$$

- Add 15 and -6 (positive > negative)

$$\begin{array}{r}
 00001111 \quad 15 \\
 +11111010 \quad + -6 \\
 \hline
 10001001 \quad 9
 \end{array}$$

Discard carry $\longrightarrow$ 1

- Add 16 and -24 (negative > positive)

$$\begin{array}{r}
 00010000 \quad 16 \\
 +11101000 \quad + -24 \\
 \hline
 11111000 \quad -8
 \end{array}$$

Sign bit is negative so negative number in 2's complement form

- Add -5 and -9 (both negative)

$$\begin{array}{r}
 11111011 \quad -5 \\
 +11110111 \quad + -9 \\
 \hline
 11110010 \quad -14
 \end{array}$$

Discard carry $\longrightarrow$ 1

Arithmetic Operations with Signed Numbers

Subtraction of Signed Numbers

- The parts of a subtraction function are:
 - Minuend - The first number
 - Subtrahend - The second number
 - Difference - The result

Subtraction is addition with the sign of the subtrahend changed.

Arithmetic Operations with Signed Numbers

Subtraction

- The sign of a positive or negative binary number is changed by taking its 2's complement
- To subtract two signed numbers, take the 2's complement of the subtrahend and add. Discard any final carry bit.

Subtraction Examples

- Find 8 minus 3.

$$\begin{array}{r}
 00001000 \\
 +11111101 \\
 \hline
 1\ 00000101
 \end{array}
 \begin{array}{r}
 8 \\
 -3 \\
 \hline
 5
 \end{array}
 \begin{array}{l}
 \text{Minuend} \\
 \text{Subtrahend} \\
 \text{Difference}
 \end{array}$$

Discard carry $\longrightarrow$

- Find 12 minus -9.

$$\begin{array}{r}
 00001100 \\
 +00001001 \\
 \hline
 00010101
 \end{array}
 \begin{array}{r}
 12 \\
 - -9 \\
 \hline
 21
 \end{array}$$

- Find -25 minus 19.

$$\begin{array}{r}
 11100111 \\
 +11101101 \\
 \hline
 1\ 11010100
 \end{array}
 \begin{array}{r}
 -25 \\
 - 19 \\
 \hline
 -44
 \end{array}$$

Discard carry $\longrightarrow$

- Find -120 minus -30.

$$\begin{array}{r}
 10001000 \\
 +00011110 \\
 \hline
 10100110
 \end{array}
 \begin{array}{r}
 -120 \\
 - -30 \\
 \hline
 -90
 \end{array}$$

Arithmetic Operations with Signed Numbers

Multiplication of Signed Numbers

- The parts of a multiplication function are:
 - Multiplicand - First number
 - Multiplier - Second number
 - Product - Result

Multiplication is equivalent to adding a number to itself a number of times equal to the multiplier.

Arithmetic Operations with Signed Numbers

There are two methods for multiplication:

- Direct addition
 - add multiplicand multiple times equal to the multiplier
 - Can take a long time if multiplier is large
- Partial products
 - Similar to long hand multiplication

The method of partial products is the most commonly used.

Arithmetic Operations with Signed Numbers

Multiplication of Signed Numbers

- If the signs are the same, the product is positive. ($+ X + = +$ or $- X - = +$)
- If the signs are different, the product is negative. ($+ X - = -$ or $- X + = -$)

Multiplication Example

- Both numbers must be in uncomplemented form
- Multiply 3 by -5.

Opposite signs, so product will be negative.

$$3_{10} = 00000011_2$$

$$-5_{10} = 11111011_2$$

00000011	Multiplicand
X 00000101	Multiplier
00000011	First partial product
+ 00000000	Second partial product
00000011	Sum of 1 st and 2 nd
+ 000011	Third partial product
00001111	Sum and Final Product

Final result is negative, so take 2's complement.

11110001 is the result which in decimal is -15.

Arithmetic Operations with Signed Numbers

Division of Signed Numbers

- The parts of a division operation are:
 - Dividend
 - Divisor
 - Quotient

$$\frac{\text{dividend}}{\text{divisor}} = \text{quotient}$$

Division is equivalent to subtracting the divisor from the dividend a number of times equal to the quotient.

Arithmetic Operations with Signed Numbers

Division of Signed Numbers

- If the signs are the same, the quotient is positive. $(+ \div + = + \text{ or } - \div - = +)$
- If the signs are different, the quotient is negative. $(+ \div - = - \text{ or } - \div + = -)$

Division Example

- Both numbers must be in uncomplemented form
- Divide 01100100 by 00110010.

Both numbers are positive so quotient will be positive.

Set the quotient to zero initially.

Subtract the divisor from the dividend by using 2's complement addition. (11001110)
Ignore the carry bit.

Subtract the divisor from the 1st partial remainder using 2's complement addition.

	01100100	Dividend
	+ 11001110	2's complement of Divisor
	1 00110010	First partial remainder
Add 1 to quotient:		$00000000 + 1 = 00000001$
	00110010	First partial remainder
	+ 11001110	2's complement of Divisor
	1 00000000	zero remainder
Add 1 to quotient:		$00000001 + 1 = 00000010$

So final quotient is 00000010 and final remainder is 00000000

BCD Code

- Binary Coded Decimal (BCD) is a widely used way to present decimal numbers in binary form.
 - Combines features of both decimal and binary systems.
 - Each digit is converted to a binary equivalent.
- BCD is *not* a number system.
 - It is a decimal number with each digit encoded to its binary equivalent.
- A BCD number is *not* the same as a straight binary number.
 - The primary advantage of BCD is the relative ease of converting to and from decimal.

BCD Code

- Convert the number 874_{10} to BCD:
 - Each decimal digit is represented using 4 bits.
 - Each 4-bit group can never be greater than 9.

8	7	4	(decimal)
↓	↓	↓	
1000	0111	0100	(BCD)

- Reverse the process to convert BCD to decimal.

9	4	3	(decimal)
↓	↓	↓	
1001	0100	0011	(BCD)

BCD Code

- Convert 0110100000111001 (BCD) to its decimal equivalent.

0110 1000 0011 1001
└───┘ └───┘ └───┘ └───┘
6 8 3 9

Divide the BCD number into four-bit groups and convert each to decimal.

BCD Code

- Convert 0110100000111001 (BCD) to its decimal equivalent.

0110 1000 0011 1001
6 8 3 9

Divide the BCD number into four-bit groups and convert each to decimal.

BCD Code

- Convert BCD 011111000001 to its decimal equivalent.

$$\begin{array}{ccc} \underbrace{0111} & 1100 & \underbrace{0001} \\ 7 & \downarrow & 1 \end{array}$$

The forbidden group represents an error in the BCD number.

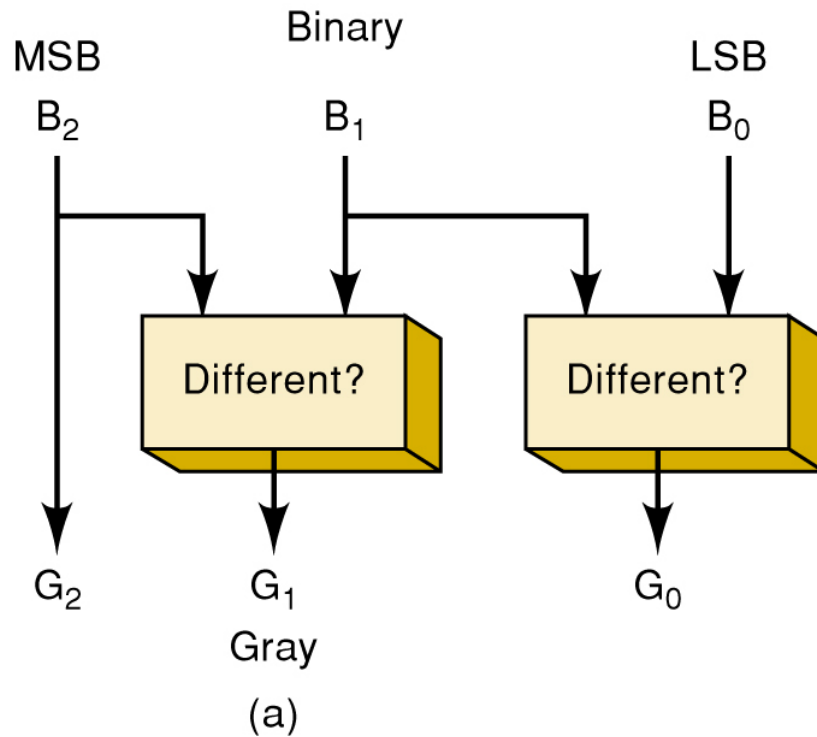
The Gray Code

- The Gray code is used in applications where numbers change rapidly.
 - Only one bit changes from each value to the next.

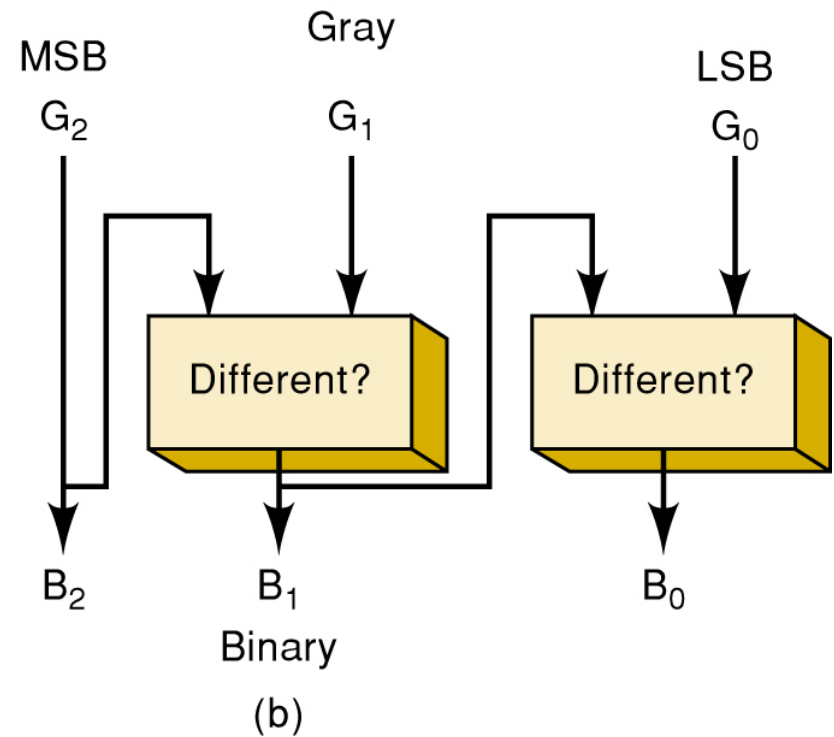
**Three bit binary
and Gray code
equivalents.**

B ₂	B ₁	B ₀	G ₂	G ₁	G ₀
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	1	1	0
1	0	1	1	1	1
1	1	0	1	0	1
1	1	1	1	0	0

The Gray Code



Binary to Gray



Gray to Binary

Putting It All Together

Decimal numbers 1 – 15 in binary, hex, BCD, Gray

Decimal	Binary	Hexadecimal	BCD	GRAY
0	0	0	0000	0000
1	1	1	0001	0001
2	10	2	0010	0011
3	11	3	0011	0010
4	100	4	0100	0110
5	101	5	0101	0111
6	110	6	0110	0101
7	111	7	0111	0100
8	1000	8	1000	1100
9	1001	9	1001	1101
10	1010	A	0001 0000	1111
11	1011	B	0001 0001	1110
12	1100	C	0001 0010	1010
13	1101	D	0001 0011	1011
14	1110	E	0001 0100	1001
15	1111	F	0001 0101	1000

The Byte, Nibble, and Word

- Most microcomputers handle and store binary data and information in groups of eight bits.
 - 8 bits = 1 byte.
 - A byte can represent numerous types of data/information.
- Binary numbers are often broken into groups of four bits.
 - Because a group of four bits is half as big as a byte, it was named a **nibble**.
- A **word** is a group of bits that represents a certain unit of information.
 - **Word size** can be defined as the number of bits in the binary word a digital system operates on.
 - PC word size is eight bytes (64 bits).

Alphanumeric Codes

- Represents characters and functions found on a computer keyboard.
 - 26 lowercase & 26 uppercase letters, 10 digits, 7 punctuation marks, 20 to 40 other characters.
- ASCII – American Standard Code for Information Interchange.
 - Seven bit code: $2^7 = 128$ possible code groups
 - Examples of use: transfer information between computers; computers & printers; internal storage.

Alphanumeric Codes

ASCII – American Standard Code for Information Interchange

Character	HEX	Decimal	Character	HEX	Decimal	Character	HEX	Decimal	Character	HEX	Decimal
NUL (null)	0	0	Space	20	32	@	40	64	.	60	96
Start Heading	1	1	!	21	33	A	41	65	a	61	97
Start Text	2	2	“	22	34	B	42	66	b	62	98
End Text	3	3	#	23	35	C	43	67	c	63	99
End Transmit.	4	4	\$	24	36	D	44	68	d	64	100
Enquiry	5	5	%	25	37	E	45	69	e	65	101
Acknowledge	6	6	&	26	38	F	46	70	f	66	102
Bell	7	7	`	27	39	G	47	71	g	67	103
Backspace	8	8	(	28	40	H	48	72	h	68	104
Horiz. Tab	9	9	)	29	41	I	49	73	i	69	105
Line Feed	A	10	*	2A	42	J	4A	74	j	6A	106
Vert. Tab	B	11	+	2B	43	K	4B	75	k	6B	107
Form Feed	C	12	,	2C	44	L	4C	76	l	6C	108
Carriage Return	D	13	-	2D	45	M	4D	77	m	6D	109
Shift Out	E	14	.	2E	46	N	4E	78	n	6E	110

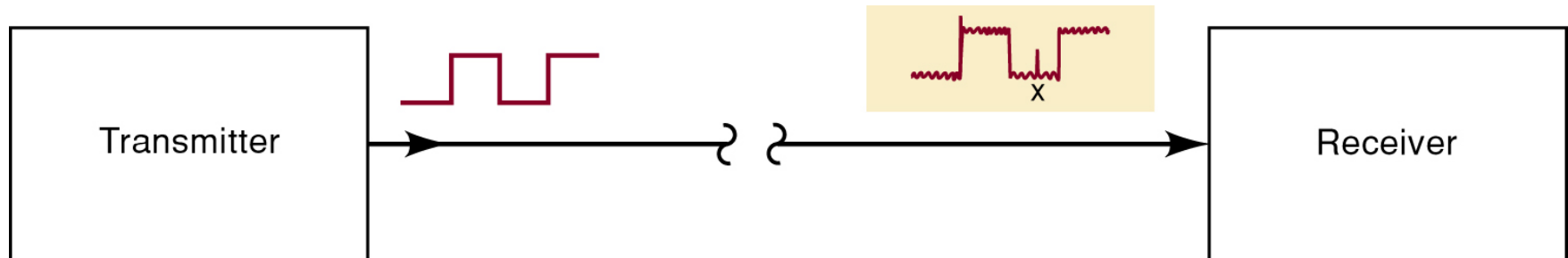
See the entire table of your textbook.

Parity Method for Error Detection

- Binary data and codes are frequently moved between locations:
 - Digitized voice over a microwave link.
 - Storage/retrieval of data from magnetic/optical disks.
 - Communication between computer systems over telephone lines, using a modem.

Parity Method for Error Detection

- Electrical noise can cause errors during transmission.
 - Spurious fluctuations in voltage or current present in all electronic systems.



- Many digital systems employ methods for error detection—and sometimes correction.
 - One of the simplest and most widely used schemes for error detection is the **parity method**.

Parity Method for Error Detection

- The parity method of error detection requires the addition of an extra bit to a code group.
 - Called the parity bit, it can be either a 0 or 1, depending on the number of 1s in the code group.
- There are two parity methods, even and odd.
 - The transmitter and receiver must “agree” on the type of parity checking used.
 - Even seems to be used more often.

Simple Parity Checking

(or One-dimension Parity Check)

- The most common and least expensive mechanism for error- detection is the simple parity check.
- The parity check is done by adding an **extra bit**, called **parity bit** to the data to make a number of **1s either even** (even parity) **or odd** (odd parity)
- While creating a frame, the sender counts the number of 1s in it and adds the parity bit in the following way:
 - If a number of 1s is even then parity bit value is 0
 - If the number of 1s is odd then parity bit value is 1.

Simple Parity Checking

(or One-dimension Parity Check)

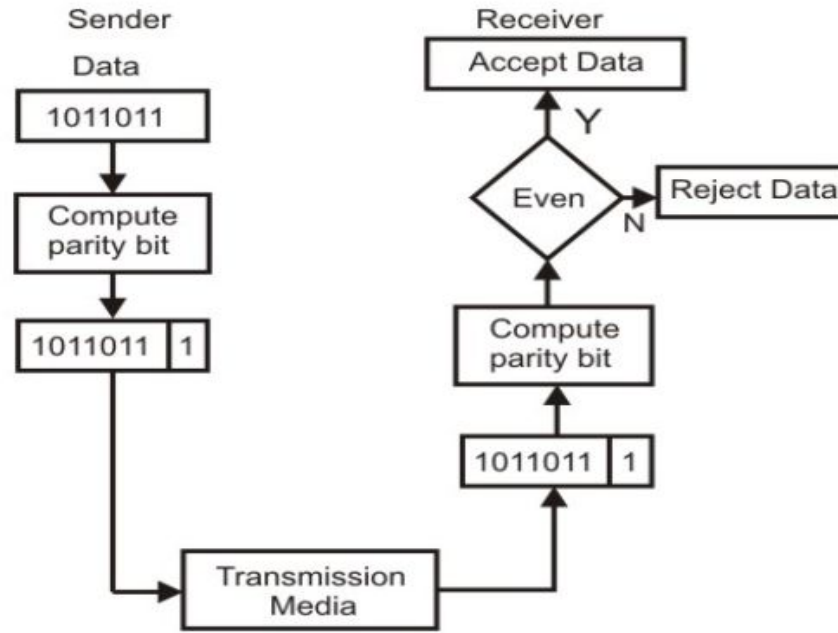


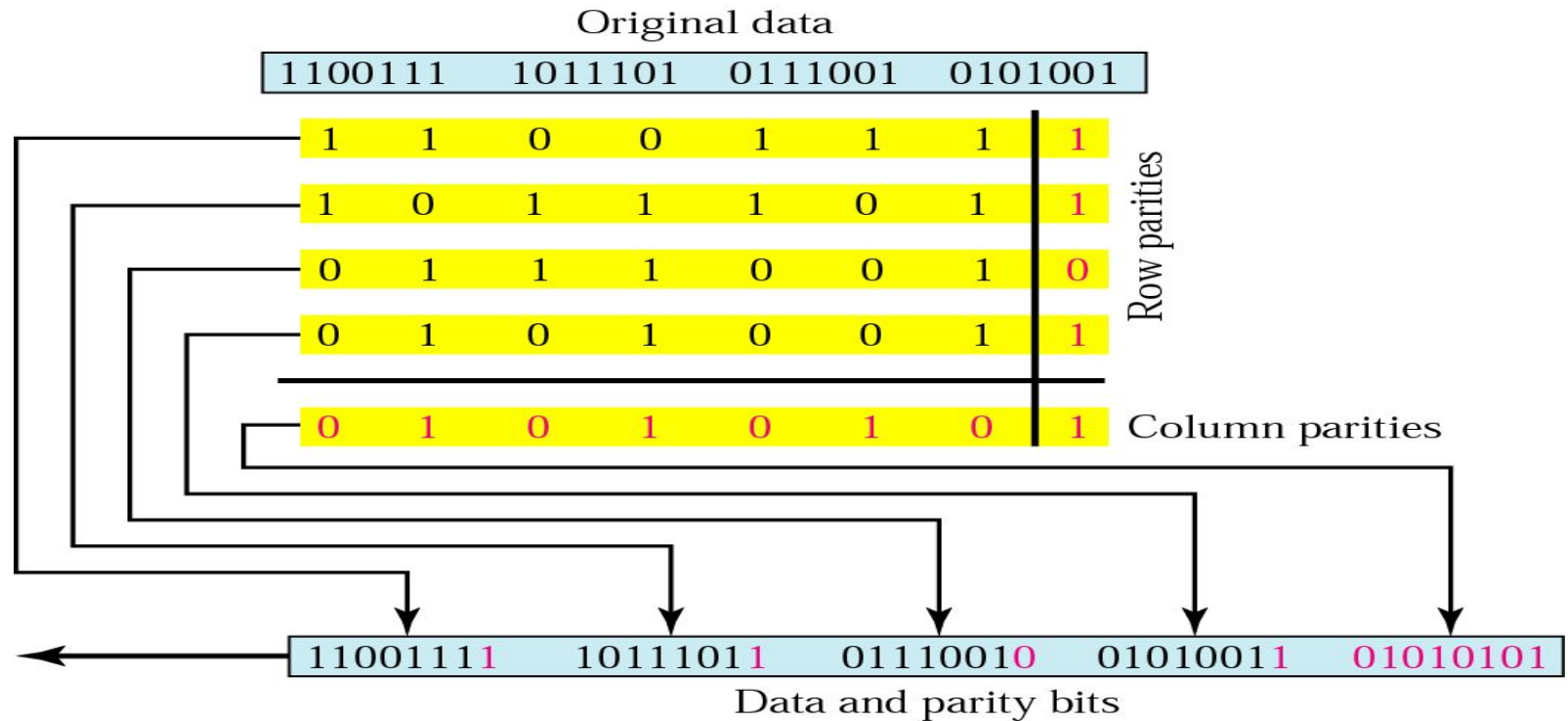
Figure 3.2.3 Even-parity checking scheme

- The parity check is suitable for **single bit error detection** only.
- A simple parity-check code can detect **odd** numbers of error.

2D Parity Checking

- Performance can be improved by using two dimensional parity check.
- Parity check bits are calculated for each row, which is equivalent to a simple parity check bit.
- Parity check bits are also calculated for all columns then both are sent along with the data.

2D Parity Checking



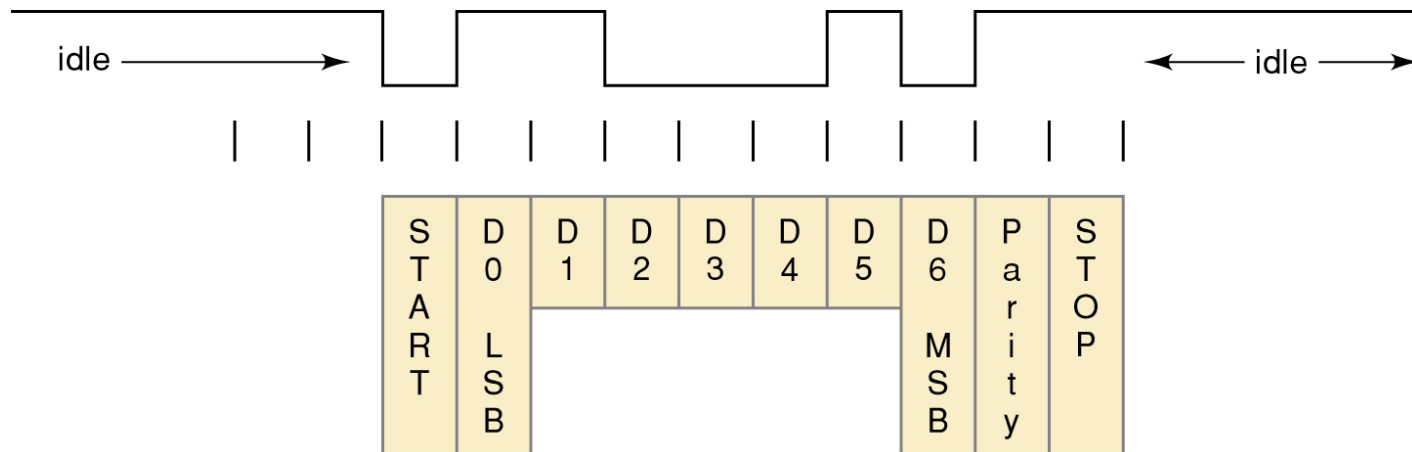
Two- Dimension Parity Checking increases the likelihood of detecting burst errors.

Applications

- When ASCII characters are transmitted there must be a way to tell the receiver a new character is coming.
 - There is often a need to detect errors in the transmission as well.
- The method of transfer is called asynchronous data communication.

Applications

- An ASCII character must be “framed” so the receiver knows where the data begins and ends.
 - The first bit must always be a start bit (logic 0).
- ASCII code is sent LSB first and MSB last.
 - After the MSB, a parity bit is appended to check for transmission errors.
 - Transmission is ended by sending a stop bit (logic 1).



Thank
You



Next topic